PASTTEL : Parallel analysiS framework for Termination and non-Termination of Lasso programs

Anissa Kheireddine[1,2 [0000−0002−5958−4069]], Souheib Baarir[2,3 [0000−0001−8140−0273]], and Hugo De Sa Pereira Pinto[1]

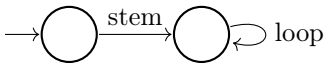
[1] Dowsers, Paris, France
first.last@dowsers.finance
[2] Sorbonne University, LIP6, Paris, France
[3] Nanterre University, Nanterre, France
first.last@lip6.fr

Abstract. Proving termination or non-termination of *lasso programs* is a challenging problem in program verification. To unify state-of-the-art approaches under a common execution framework, we present PASTTEL, a modular and generic parallel portfolio framework for termination and non-termination analysis of lasso programs. PASTTEL is designed to: (1) facilitate the integration of new analysis algorithms into the portfolio, (2) execute registered strategies concurrently, and (3) act as a self-contained library component that can be seamlessly embedded into any external project requiring (non-)termination analysis. Initial experiments demonstrate that an instantiation of PASTTEL performs competitively against state-of-the-art tools.

1 Introduction

Proving the termination of a program is one of the most fundamental correctness properties in computer science, and is known to be undecidable in the general case. A practical way to approach this problem is to search for explicit termination or non-termination arguments. The existence of a certain termination argument — such as a linear ranking function — is decidable [5,7,26,30,35] and implies termination. However, if no linear ranking function can be found, we cannot conclude non-termination. Conversely, the existence of a non-termination argument, e.g., a linear recurrence set [24] or a geometric non-termination argument [31], is decidable and implies non-termination, but a failure to find one cannot conclude termination.

These analyses are commonly applied to *lasso programs*: execution traces consisting of a single while loop preceded by straight-line code, whose control flow graph takes the characteristic shape of a lasso.



In typical use cases, a lasso program is extracted from a control flow graph to represent a candidate infinite execution path, as encountered in CEGAR-based termination synthesis [13,14,15,19].

We introduce PaSTTeL[4], a parallel analysis framework for the termination and non-termination of lasso programs. PaSTTeL unifies state-of-the-art techniques within a concurrent portfolio that combines both termination and non-termination approaches. Our main contributions are as follows:

- We propose a modular and generic framework for implementing termination and non-termination strategies, designed to facilitate their integration into a unified portfolio.
- We implement several state-of-the-art constraint-based techniques, including ranking function synthesis [30] and geometric non-termination arguments [31].
- We evaluate an instance of our framework on the SV-COMP 2025[5] benchmark suite, comparing its performance with the state-of-the-art Ultimate LassoRanker[6]. The results show that the modularity and genericity of PaSTTeL do not compromise performance, while its C++-based implementation yields a significant speedup in execution time.

Beyond its standalone use, PaSTTeL is designed as a self-contained library that any external project can integrate to leverage (non-)termination analysis capabilities without knowledge of its internal design.

The remainder of this paper is organized as follows. Section 2 surveys the state-of-the-art termination and non-termination analysis tools and positions PaSTTeL among them. Section 3 introduces the formal definition of lasso programs and describes the corresponding input format of PaSTTeL. Section 4 describes the architecture of the framework and the concurrent orchestration of the analysis strategies. Section 5 showcases an instantiation of PaSTTeL based on Ultimate LassoRanker strategies. We conclude with an empirical evaluation of the tool in Section 6.

2 PaSTTeL vs. the State-of-the-Art

To motivate the design of PaSTTeL and highlight its originality, we first provide an overview of the state-of-the-art tools for (non-)termination synthesis.

Existing tools. Termination analysis is a very active field of research, and most tools operate at the whole-program level, iteratively extracting and analyzing individual lassos as part of their verification strategy. Among these, PROTON [32,33] integrates a fine-tuned LLM for ranking function synthesis, achieving a gold medal at SV-COMP2025[5] in the Termination category. Ultimate

[4] `https://github.com/akheireddine/PaSTTeL.git`
[5] `https://sv-comp.sosy-lab.org/2025/`
[6] `https://www.ultimate-pa.org/lasso_ranker/`

Automizer [19,25], the overall winner of SV-COMP 2025, implements a CEGAR loop combining Ultimate LassoRanker with automata-theoretic techniques: termination arguments for individual lassos are generalized into Büchi automata representing all traces covered by the proof, which are then removed from the program via complementation and intersection. CPAchecker [2] takes a different approach, extracting candidate lassos via predicate-based reachability analysis, but likewise delegates their (non-)termination analysis to Ultimate LassoRanker. Beyond these, other whole-program tools such as AProVE [23,27], T2 [10], and VeryMax [6,28] also address termination through their own dedicated strategies.

While the tools above operate at the program level, a smaller subset focuses specifically on single-lasso analysis, which is directly relevant to our work. Ultimate LassoRanker [26,29,30] is a Java library integrated into Ultimate Automizer verification framework[7]. It extracts and analyzes linear lasso programs then provides termination proofs based on ranking function templates (linear, lexicographic, nested, and piecewise-affine) with linear inductive supporting invariants, and represents the state-of-the-art for single-lasso termination analysis over linear arithmetic. For non-termination, Ultimate LassoRanker [31] synthesizes *Geometric Non-Termination Arguments* (GNTAs) for linear deterministic conjunctive lassos, applying fixpoint checking first and GNTA only if the former fails. Similarly, iRankFinder [5] synthesizes linear, lexicographic-linear, and multiphase-linear ranking functions for multi-path loops. On the non-termination side, LoAT [21,22] (Loop Acceleration Tool) proves non-termination via loop acceleration, computing closed-form summaries of loop iterations to detect reachable non-terminating behaviors. Finally, CompACT [37,38] proves termination of linear loops through compositional abstraction using categorical deterministic affine transition systems.

Positioning PaSTTeL among existing tools. The approaches surveyed above share a common limitation: modularity is rarely a design goal, and adding a new termination or non-termination technique typically requires deep knowledge of the tool's internals. Moreover, concurrent portfolio execution remains largely absent from existing tools. To our knowledge, AProVE is the only tool that comes close to this objective, orchestrating three backends concurrently: T2-based recurrent sets, LoAT-based acceleration, and direct SMT encoding. However, AProVE is not open-source, which limits both its reusability and extensibility.

While Ultimate LassoRanker is open-source, its analyses are applied sequentially: on the termination side, ranking templates are tried one after another and on the non-termination side, fixpoint checking is attempted first, with GNTA executed only if the former fails. Furthermore, Ultimate LassoRanker does not provide a modular interface for registering new strategies; its termination and non-termination analyses are tightly coupled to the internal architecture of the Ultimate framework, making standalone reuse non-trivial.

[7] `https://www.ultimate-pa.org/automizer/`

PaSTTeL is designed to address these limitations. It is an open-source, generic, and modular C++ framework that also exposes a library interface, enabling any external project to embed and leverage (non-)termination analysis capabilities as a self-contained component. New strategies can be integrated by implementing a single interface, making it straightforward to enrich the portfolio. Unlike AProVE, the portfolio is fully extensible. Unlike LassoRanker, all registered strategies are executed concurrently, so that the first conclusive result — whether a termination or non-termination proof — is immediately returned without waiting for the remaining strategies to complete. Beyond the approaches currently integrated, the modular design of PaSTTeL makes it straightforward to incorporate any existing technique. As a proof of concept, Section 5 presents an instantiation of PaSTTeL based on the techniques underlying Ultimate LassoRanker.

3 Lasso Program

A variety of techniques have been proposed for reasoning about lasso programs, including constraint-based methods [1,5,7,8,12,26,31,35], abstract interpretation [16,36], and algebraic approaches based on eigenvalue analysis [9]. As this work focuses primarily on constraint-based methods, we now introduce the formal definitions of (*linear*) lasso programs and the notion of termination considered throughout the paper.

We denote by $\mathbf{x} = (x_1, \ldots, x_n)^\top \in \mathbb{R}^n$ the vector of n program variables, and by $\mathbf{x}' = (x'_1, \ldots, x'_n)^\top \in \mathbb{R}^n$ the variables of the next state.

To abstract away from program syntax, we define a lasso program directly by the two relations that generate its execution sequences.

Definition 1 (Lasso Program [26,31]). *Given a set of states Σ, a lasso program*

$$P = (\tau_{stem}, \tau_{loop})$$

consists of two relations $\tau_{stem} \subseteq \Sigma \times \Sigma$ and $\tau_{loop} \subseteq \Sigma \times \Sigma$, called the stem *and the* loop *of P, respectively.*

An execution of P is a possibly infinite sequence of states $\sigma_0, \sigma_1, \ldots$ such that $(\sigma_0, \sigma_1) \in \tau_{stem}$ and $(\sigma_i, \sigma_{i+1}) \in \tau_{loop}$ for all $i \geq 1$. The program P is terminating if it admits no infinite execution.

A loop program is the special case where $\tau_{stem} \equiv \mathit{true}$.

Linear Arithmetic. The expression $A \cdot \mathbf{x} \leq \mathbf{b}$ denotes the conjunction of linear constraints with $(n, m) \in \mathbb{N}^2$, $A \in \mathbb{R}^m \times \mathbb{R}^n$ and $\mathbf{b} \in \mathbb{R}^m$,

$$\bigwedge_{j=0}^{m} \left(\sum_{i=0}^{n} a_{ij} \cdot x_i \right) \leq b_j.$$

```
int x = 10;
int y = 0;
while (x > y) {
    x = x / 2;
    y = y + 2;
}
```

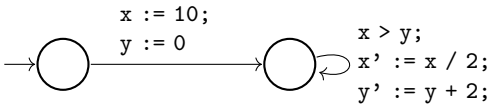


Fig. 1: The control flow graph of the left-side lasso program is defined by $\tau_{\text{stem}} =$ `x := 10; y := 0` and $\tau_{\text{loop}} =$ `x > y; x' := x / 2; y' := y + 2;`

Linear relation. A linear relation τ, is a relation defined by a conjunction of linear inequalities over $\mathbf{x}$ and $\mathbf{x}'$ with $(n, m) \in \mathbb{N}^2$, $A \in \mathbb{R}^m \times \mathbb{R}^{2n}$ and $\mathbf{b} \in \mathbb{R}^m$:

$$\tau = \{(\mathbf{x}, \mathbf{x}') \in \mathbb{R}^n \times \mathbb{R}^n \mid A \begin{pmatrix} \mathbf{x} \\ \mathbf{x}' \end{pmatrix} \leq \mathbf{b}\}$$

Definition 2 (Linear Lasso Program [26,31]). *A linear lasso program is a pair*

$$P = (\tau_{stem}, \tau_{loop})$$

where $\tau_{stem}, \tau_{loop} \subseteq \mathbb{R}^n \times \mathbb{R}^n$ *are linear relations over the current-state variables* $\mathbf{x}$ *and next-state variables* $\mathbf{x}'$.

Figure 1 illustrates these definitions on a concrete example. The program in left-side of Figure 1 consists of a straight-line initialization followed by a single while loop. Its formal representation in right-side of Figure 1 exhibits the characteristic lasso shape: a stem encoding the initial assignments `x := 10; y := 0`, and a loop body encoding the transition `x' := x / 2; y' := y + 2;` guarded by `x > y`.

Linearization of Lasso Programs

Some (non-)termination analysis strategies operate exclusively on linear lasso programs [26,29,30,31]. When the input lasso contains non-linear constructs (Boolean variables, division, modulo, ...), a pre-processing linearization phase is initiated[6]. For instance, Boolean variables are replaced by integer comparisons over a fresh auxiliary variable v, where *true* is encoded as $v \geq 1$ and *false* as $v \leq 0$.

When full linearization is not achievable — for instance, when an instruction translating a product of two free variables $x \times y$ appears — three strategies may be adopted[6]. First, *over-approximation*: the non-linear constraint is replaced by $\top$, making the constraint trivially satisfied; termination analyses may then become too permissive and fail to produce a witness. Second, *under-approximation*: the constraint is replaced by $\bot$, making it unsatisfiable; non-termination analyses may then fail to retrieve a valid non-terminating execution. Third, an *exception* is raised to signal that linearization could not be completed. All three strategies are implemented in PASTTEL[4], and the desired behavior can be specified via a configuration parameter.

4 Architecture of the Framework

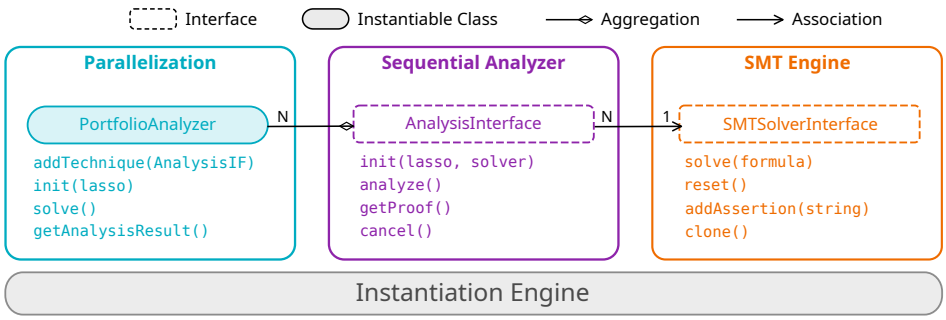


Fig. 2: The architecture of PaSTTeL

The performance of existing termination and non-termination analysis strategies heavily depends on implementation choices such as the programming language or the libraries used (*e.g.*, C++ vs. Java). This makes it difficult to compare strategies fairly without introducing a technological bias, or at best, fully reimplementing them to eliminate it. Moreover, integrating new strategies on top of existing ones remains a non-trivial task.

PaSTTeL is designed to address these issues. The architecture of PaSTTeL is structured around three core components: a sequential analyzer (`AnalysisInterface`); an SMT engine (`SMTSolverInterface`); and a parallelization layer (`PortfolioAnalyzer`), as illustrated in Figure 2). Any instantiation of the framework must provide concrete implementations of these three components.

SMT engine: `SMTSolverInterface`. This interface allows different SMT solvers to be plugged into the framework interchangeably. PaSTTeL currently supports Z3 [18] and CVC5 [3]. The solver instance is passed to each sequential analyzer and used by the constraint-based analysis techniques.

Sequential Analyzer: `AnalysisInterface`. This adapter defines the basic functions expected from a sequential termination or non-termination analyzer. The `LassoProgram` data structure used as input is described in Figure 3. The main methods of this interface are:

- `void init(LassoProgram* lasso)`: initializes the analyzer with the input lasso program.
- `AnalysisResult analyze()`: launches the analysis using the analyzer's own SMT solver instance. Returns `TERMINATING`, `NON_TERMINATING`, or `UNKNOWN`.
- `ProofCertificate getProof()`: returns a proof certificate for the (non-)termination result in plain text format.
- `void cancel()`: interrupts the current search initiated by the caller.

```
class LassoProgram {
public:
    LinearTransition stem;
    LinearTransition loop;
    // Program variables names
    std::vector<std::string> program_vars;
    // Program variables sorts
    std::map<std::string, std::string> var_sorts;
    ...
};
```

Fig. 3: `LassoProgram` class

Parallelization: `PortfolioAnalyzer`. This component manages the threads deployed for the concurrent execution of termination and non-termination strategies on the same input lasso program. Its main methods are:

- `void setNumThread(int n)`: sets the thread pool size, i.e., the maximum number of concurrently running threads
- `void addTechnique(AnalysisInterface* technique)`: registers an analysis technique into the portfolio.
- `void solve()`: launches the concurrent search across all registered strategies.
- `ProofCertificate join(int timelimit)`: blocks until one technique returns a conclusive result or the time limit is reached. If the result differs from `UNKNOWN`, the corresponding proof certificate is returned.

Algorithm 1: `init-PaSTTeL` function

Input: *lasso*: linear lasso program, n: number of threads, *solverType*: {`Z3`, `CVC5`}

```
1 solvers ← Create n of solverType from SMTSolverInterface;
2 portfolio ← Create PortfolioAnalyzer(lasso,n);
3 portfolio.addTechnique(new TerminationSA(...), solvers[0]);
4 portfolio.addTechnique(new NonTerminationSA(...), solvers[1]);
5 portfolio.solve();
6 portfolio.join(timelimit);
7 print(portfolio.getAnalysisResult());
8 if portfolio.getAnalysisResult() ≠ UNKNOWN then
9 |   print(portfolio.getProof());
```

Engine Instantiation. To create a particular instance of PaSTTeL, the user has to adapt the `init-PaSTTeL` function (Algorithm 1), which serves as the

entry point for instantiating and binding all components of the framework. Its parameters expose the key configuration points, allowing any external tool to embed PaSTTeL as a self-contained library with the desired configuration.

First, the n concrete solver instances of type *solverType* (inheriting from `SMTSolverInterface`) are instantiated (line 1). The portfolio analyzer is then instantiated with a reference to the *lasso* structure (line 2). Analysis strategies (inherited from `SequentialAnalyzer`) are registered into the portfolio (lines 3–4), each linked to a dedicated solver instance. The concurrent search is initiated via `solve` (line 5), and the main thread blocks until a conclusive result is found or the time limit is reached (line 6). The final result is retrieved (line 7), along with the proof certificate if one was produced (lines 8–9).

5 PaSTTeL "à la Ultimate LassoRanker"

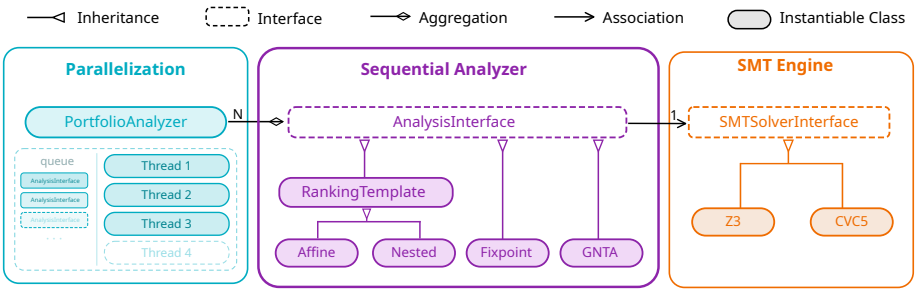


Fig. 4: P-ULR instantiation architecture

To validate the generic aspect of our framework, we selected the techniques underlying Ultimate LassoRanker (ULR) — the state-of-the-art tool for linear lasso-level analysis and overall winner of SV-COMP 2025 — as a proof of concept. We implemented P-ULR, an instantiation of PaSTTeL that replicates the core (non-)termination strategies of ULR. Each technique is realized as an independent analyzer registered into the portfolio and executed concurrently with the others. The resulting instantiation is illustrated in Figure 4.

Input Format & Linearization. While PaSTTeL operates directly on linear lasso program structures (see Section 4), P-ULR provides a JSON-based input format (see Figure 5) as a convenience layer, designed to be generic and tool-agnostic. Formulas are encoded following the SMT-LIB standard [4], with support for Single Static Assignment (SSA) form [17]. Each input file specifies the program variables and their types, together with the input and output variables of both τ_{stem} and τ_{loop}[8].

[8] Support for additional fields, including axioms and uninterpreted functions, is currently under development.

```
{
  "program_vars": [
      {
        "x": "Int",
        "y": "Int"
      }],
  "stem": [
      {
        "formula":  "(and (= x_0 10)
                          (= y_0 0))",
        "in_vars":  {},
        "out_vars": { "x": "x_0", "y": "y_0" }
      }],
  "loop": [
      {
        "formula":  "(and (> x_1 y_1)
                          (= x_2 (div x_1 2))
                          (= y_2 (+ y_1 2)))",
        "in_vars":  { "x": "x_1", "y": "y_1" },
        "out_vars": { "x": "x_2", "y": "y_2" }
      }]
}
```

Fig. 5: Input format of a the lasso program in Figure 1

P-ULR consists of approximately 7,000 lines of C++ code, built on top of the 3,500 lines of the core framework. Given a lasso program in the JSON-based SMT-LIB format, P-ULR first linearizes it if necessary, then dispatches the analysis to the registered strategies. Since all implemented components inherit from the framework's abstract interfaces, they are fully reusable and can be extended or complemented by additional techniques, such as new ranking function templates, without modifying the existing codebase.

Termination. P-ULR provides a constraint-based synthesis of termination arguments for linear lasso programs using linear ranking templates [29,30]. This approach generalizes constraint-based termination analysis by providing a unified framework for synthesizing ranking functions with affine-linear components, encompassing *nested*, *piecewise*, *parallel*, and *lexicographic* ranking functions, which can further be combined to form a more powerful composite templates. Rather than using Farkas' lemma [20], the approach relies on Motzkin's transposition theorem [34] for two reasons: (1) to transform the generated $\exists\forall$-constraint into an $\exists$-constraint directly solvable by an SMT solver, and (2) to handle ranking templates that require both strict and non-strict inequalities. The method is extended to the general setting of linear lasso programs by using affine-linear inductive invariants to extract from the stem the information that is crucial for the termination proof [7,11,26]. For instance, on the example of Figure 1, the synthesized ranking function is $f(x, y) = 2x - 2y + 1$, with supporting invariant

$y + 1 \geq 0$.
Currently, P-ULR implements *affine* and *nested* ranking templates.

Non-Termination. P-ULR implements a constraint-based synthesis of non-termination arguments for linear lasso programs, integrating two complementary approaches: *fixpoint* checking and *Geometric Non-Termination Arguments* (GNTA) [31]. A GNTA represents an infinite execution as a pointwise sum of geometric series: for $k > 1$ the k-th state is: $\mathbf{x_1} + \sum_{i=0}^{k} \lambda^i \cdot \mathbf{y}$, where $\mathbf{x}_1$ is the program state before the first loop iteration, $\mathbf{y}$ is the initial direction of the execution, and λ is the speed at which the execution continues to move in that direction. The approach is complete for lassos with non-negative eigenvalues and reduces the search to an existential constraint over non-linear integer arithmetic, solvable by an SMT solver. *Fixpoint* checking, in contrast, searches for a state $\mathbf{x}$ such that $(\mathbf{x}, \mathbf{x}) \in \tau_{\text{loop}}$, i.e., a state from which the loop iterates indefinitely without change. As described in Section 2, ULR applies these two methods sequentially; P-ULR instead runs both methods concurrently, without blocking the GNTA analysis on the outcome of *fixpoint* checking.

Limitations. The current version of P-ULR has several limitations. First, the linearization step does not yet cover all non-linear cases: in particular, array operations may require special treatment, including the generation of supporting invariants for array variables, which is not currently supported. Second, at the framework level, the `LassoProgram` structure currently provides better support for linear formulas; non-linear formulas are passed as-is to the registered strategies, which is valid for strategies that do not require linearization such as *fixpoint*. Third, at the strategy level, P-ULR currently supports linear rational arithmetic (QF_LRA) for termination synthesis and linear integer arithmetic (QF_LIA) for GNTA synthesis. Finally, simplification of synthesized ranking function coefficients and formula optimization have not yet been implemented. Extensions to QF_NRA and QF_NIA, as well as the improvements mentioned above, are left for future work.

6 Numerical comparison

Experimental Setup. The experiments were conducted on a machine with 16 Intel Core i5-14400T processors and 30 GB of memory. We used C program instances drawn from the SV-COMP 2025[5] and TermCOMP 2025[9] competitions, as well as Boogie instances from the Ultimate[6] repository.

To obtain the benchmark, Ultimate BuchiAutomizer — which relies on ULR as its internal lasso-level analysis library, also used by Ultimate Automizer — was run on each input program with a time-out of 600 seconds, during which the lasso programs generated internally by Ultimate BuchiAutomizer and analyzed by ULR were serialized to JSON files. The time spent

[9] `https://termination-portal.org/wiki/TPDB`

Tool	#Terminating	Time (s)	#Non-Terminating	Time (s)
ULR-Baseline		1309.64		47.13
P-ULR-Seq	3531	492.61	1088	1.78
P-ULR-Par4		**470.74**		**1.62**

Table 1: Comparison of P-ULR with ULR on 4619 lasso programs. For each tool, we report the number of instances proved terminating and their accumulated solving time, followed by the number of instances proved non-terminating and their accumulated solving time, both in seconds.

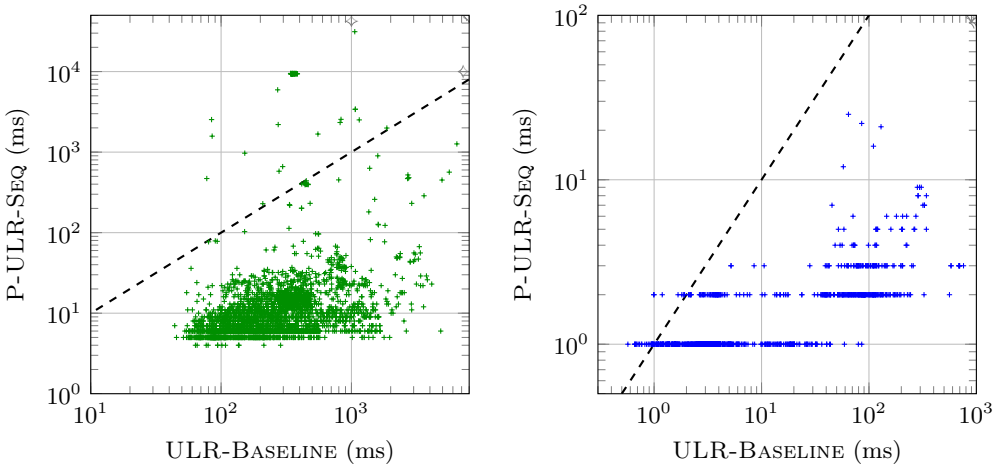
by ULR on each individual strategy was logged directly into the corresponding lasso file. P-ULR was then run on the same benchmark with a time-out of 600 seconds per instance.

Sequential Comparison. To ensure a fair comparison between P-ULR and ULR, we disabled the parallel execution of P-ULR and ran it in sequential mode, denoted P-ULR-Seq. On the other hand ULR, renamed as ULR-Baseline, executes its strategies sequentially in a fixed order: *fixpoint* checking is launched first; if no conclusive result is obtained, *GNTA* is run; and if still inconclusive, termination synthesis via ranking templates is performed in the following order: *affine*, *lexicographic*, *multiphase*, *nested*, *piecewise*, *parallel*, and *composed lexicographic*. P-ULR-Seq follows the same strategy order. Both tools were evaluated using Z3 as the underlying SMT solver.

Table 1 reports the results of analyzing 4619 lasso programs, shown in the first two rows of the table. For each tool, we report the number of instances proved terminating and non-terminating, along with the accumulated solving time over the commonly solved instances. Overall, P-ULR-Seq achieves a lower solving time than ULR-Baseline, with a total gain of 862 s (∼14 minutes). Notably, the accumulated time for non-termination synthesis is reduced by a factor of 26, from 47.13 seconds down to 1.78 seconds.

The scatter plots in Figures 6a and 6b illustrate the same results on a logarithmic time scale.

– Regarding terminating instances (Figure 6a), P-ULR-Seq significantly reduces the time required to find a ranking function compared to ULR-Baseline. The majority of lasso programs admit *affine* ranking function as their termination proof, while a smaller subset requires *nested*, *lexicographic* or *multiphase* ranking templates. Although P-ULR-Seq currently supports only *affine* and *nested* templates, this does not prevent it from producing termination arguments for the benchmark instances, since these two templates suffice for the majority of cases.
  However, both tools do not always synthesize identical ranking functions. ULR applies an additional partitioning step that decomposes the problem into smaller sub-problems, enabling the synthesis of simpler and more readable ranking functions. P-ULR, which does not incorporate this step, produces ranking functions with larger coefficients and is currently restricted to



(a) Terminating lasso programs (b) Non-Terminating lasso programs

Fig. 6: Scatter plots of P-ULR-Seq against ULR-Baseline

affine and *nested* templates (a limitation discussed in Section 5). Furthermore, we observe that this partitioning step also reduces the execution time of ULR-Baseline on terminating instances, as reflected by the data points above the dashed $y = x$ diagonal.

– Regarding non-terminating instances (Figure 6b), two clusters are visible. The bottom-left cluster corresponds to instances solved via fixpoint-based non-termination proofs, where the solving times of both tools are closely aligned. The upper-right cluster corresponds to instances solved via geometric non-termination arguments (GNTA), where P-ULR-Seq demonstrates a clear advantage in handling more complex non-termination proofs.

Parallel Comparison. We now compare both tools in their full execution mode. P-ULR is run with 4 concurrent threads, referred to as P-ULR-Par4, each running an independent `SequentialAnalyzer` instantiated with one of the following strategies: *fixpoint*, *GNTA*, *affine*, *nested* (iterating from *2-nested* to *5-nested*) ranking templates.

The results are reported in the third row of Table 1. P-ULR-Par4 reduces the total execution time by a factor of 2.8, achieving a gain of 884.4 seconds over ULR-Baseline. However, compared to P-ULR-Seq, the parallel execution yields little additional speedup. This is explained by the composition of the benchmark: 98% of lasso programs admit an *affine* ranking function, which P-ULR-Seq already resolves first in its sequential order, leaving limited opportunity for parallelism to bring further improvement on this benchmark.

Z3 vs. CVC5. To further exploit the flexibility of PaSTTeL's architecture, we ran the same benchmark using CVC5 as the underlying SMT solver. Figure 7

Category	Z3	CVC5
#TERM	470.74	**317.83**
#NONTERM	**1.62**	3.01

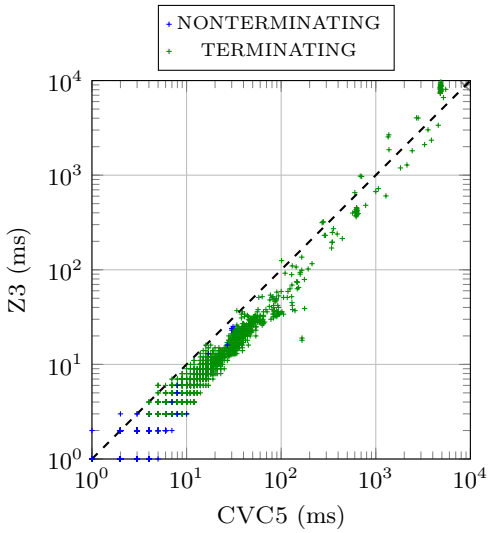


Fig. 7: Z3 vs. CVC5 SMT solver comparison on P-ULR-Par4: total computation times and per-instance scatter plot.

reports the accumulated execution times for both solvers and the associated scatter plot, comparing P-ULR-Par4 with Z3 against P-ULR-Par4 with CVC5. Both solvers achieve comparable overall performance; however, CVC5 is faster on terminating instances, reducing the termination synthesis time by 153 seconds, whereas Z3 performs better on non-terminating instances. This experiment also highlights a promising direction: since PaSTTeL supports multiple SMT solvers within the same portfolio, concurrency can be further increased by assigning different solvers to different analysis strategies, potentially combining their respective strengths.

7 Conclusion

We presented PaSTTeL, a modular and tool-agnostic framework that unifies state-of-the-art termination and non-termination analyses for lasso programs into a concurrent portfolio. As a proof of concept, we instantiated the framework with the techniques underlying Ultimate LassoRanker. The experimental results show that the execution provided by PaSTTeL reduces the overall analysis time.

Several directions are planned as future work. First, we intend to extend the linearization pre-processing step to cover a broader class of non-linear programs, starting with the full set of lasso programs produced by Ultimate Automizer, and progressively supporting additional non-linear constructs such as bit-vector arithmetic. Second, on the non-termination side, we plan to integrate recurrence

set synthesis as an additional strategy, further enriching the portfolio. Finally, we aim to address the limitations discussed above.

References

1. Roberto Bagnara, Fred Mesnard, Andrea Pescetti, and Enea Zaffanella. A new look at the automatic synthesis of linear ranking functions. *Information and Computation*, 215:47–67, 06 2012. doi:10.1016/j.ic.2012.03.003.
2. Daniel Baier, Dirk Beyer, Po-Chun Chien, Marek Jankola, Matthias Kettl, Nian-Ze Lee, Thomas Lemberger, Marian Lingsch-Rosenfeld, Martin Spiessl, Henrik Wachowitz, and Philipp Wendler. Cpachecker 2.3 with strategy selection: (competition contribution). In *Tools and Algorithms for the Construction and Analysis of Systems: 30th International Conference, TACAS 2024, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2024, Luxembourg City, Luxembourg, April 6–11, 2024, Proceedings, Part III*, page 359–364, Berlin, Heidelberg, 2024. Springer-Verlag. doi:10.1007/978-3-031-57256-2_21.
3. Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar. cvc5: A versatile and industrial-strength smt solver. In *Tools and Algorithms for the Construction and Analysis of Systems: 28th International Conference, TACAS 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2–7, 2022, Proceedings, Part I*, page 415–442, Berlin, Heidelberg, 2022. Springer-Verlag. doi:10.1007/978-3-030-99524-9_24.
4. Clark Barrett, Pascal Fontaine, and Cesare Tinelli. The SMT-LIB Standard: Version 2.7. Technical report, Department of Computer Science, The University of Iowa, 2025. Available at www.SMT-LIB.org.
5. Amir M. Ben-Amram and Samir Genaim. On the linear ranking problem for integer linear-constraint loops. *CoRR*, abs/1208.4041, 2012. URL: http://arxiv.org/abs/1208.4041, arXiv:1208.4041.
6. Cristina Borralleras, Marc Brockschmidt, Daniel Larraz, Albert Oliveras, Enric Rodríguez-Carbonell, and Albert Rubio. Proving termination through conditional termination. In Axel Legay and Tiziana Margaria, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, pages 99–117, Berlin, Heidelberg, 2017. Springer Berlin Heidelberg.
7. Aaron R. Bradley, Zohar Manna, and Henny B. Sipma. Linear ranking with reachability. In Kousha Etessami and Sriram K. Rajamani, editors, *Computer Aided Verification*, pages 491–504, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.
8. Aaron R. Bradley, Zohar Manna, and Henny B. Sipma. *Termination analysis of integer linear loops*, page 488–502. Springer-Verlag, Berlin, Heidelberg, 2005. URL: https://doi.org/10.1007/11539452_37.
9. Mark Braverman. Termination of integer linear programs. In Thomas Ball and Robert B. Jones, editors, *Computer Aided Verification*, pages 372–385, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg.
10. Marc Brockschmidt, Byron Cook, Samin Ishtiaq, Heidy Khlaaf, and Nir Piterman. T2: Temporal property verification. In Marsha Chechik and Jean-François Raskin, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, pages 387–393, Berlin, Heidelberg, 2016. Springer Berlin Heidelberg.

11. Michael Colón, Sriram Sankaranarayanan, and Henny B. Sipma. Linear invariant generation using non-linear constraint solving. In *International Conference on Computer Aided Verification*, 2003. URL: `https://api.semanticscholar.org/CorpusID:36525`.
12. Michael Colón and Henny Sipma. Synthesis of linear ranking functions. In *Proceedings of the 7th International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, TACAS 2001, page 67–81, Berlin, Heidelberg, 2001. Springer-Verlag.
13. Byron Cook, Heidy Khlaaf, and Nir Piterman. On automation of ctl* verification for infinite-state systems. In Daniel Kroening and Corina S. Păsăreanu, editors, *Computer Aided Verification*, pages 13–29, Cham, 2015. Springer International Publishing.
14. Byron Cook, Eric Koskinen, and Moshe Vardi. Temporal property verification as a program analysis task. In *Proceedings of the 23rd International Conference on Computer Aided Verification*, CAV'11, page 333–348, Berlin, Heidelberg, 2011. Springer-Verlag.
15. Byron Cook, Andreas Podelski, and Andrey Rybalchenko. Abstraction refinement for termination. In Chris Hankin and Igor Siveroni, editors, *Static Analysis*, pages 87–101, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.
16. Patrick Cousot and Radhia Cousot. An abstract interpretation framework for termination. *SIGPLAN Not.*, 47(1):245–258, January 2012. `doi:10.1145/2103621.2103687`.
17. Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. Efficiently computing static single assignment form and the control dependence graph. *ACM Transactions on Programming Languages and Systems*, 13(4):451–490, 1991.
18. Leonardo De Moura and Nikolaj Bjørner. Z3: an efficient smt solver. In *Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, TACAS'08/ETAPS'08, page 337–340, Berlin, Heidelberg, 2008. Springer-Verlag.
19. Daniel Dietsch, Matthias Heizmann, Vincent Langenfeld, and Andreas Podelski. Fairness modulo theory: A new approach to LTL software model checking. In Daniel Kroening and Corina S. Pasareanu, editors, *Computer Aided Verification - 27th International Conference, CAV 2015, San Francisco, CA, USA, July 18-24, 2015, Proceedings, Part I*, volume 9206 of *Lecture Notes in Computer Science*, pages 49–66. Springer, 2015. `doi:10.1007/978-3-319-21690-4\_4`.
20. Julius Farkas. Theorie der einfachen ungleichungen. *Journal für die reine und angewandte Mathematik (Crelles Journal)*, 1902(124):1–27, 1902.
21. Florian Frohn and Jürgen Giesl. Proving non-termination via loop acceleration. *CoRR*, abs/1905.11187, 2019. URL: `http://arxiv.org/abs/1905.11187`, `arXiv:1905.11187`.
22. Florian Frohn and Jürgen Giesl. Proving non-termination and lower runtime bounds with loat (system description), 2022. URL: `https://arxiv.org/abs/2202.04546`, `arXiv:2202.04546`.
23. Jürgen Giesl, Cornelius Aschermann, Marc Brockschmidt, Fabian Emmes, Florian Frohn, Carsten Fuhs, Jera Hensel, Carsten Otto, Martin Plücker, Peter Schneider-Kamp, Thomas Ströder, Stephanie Swiderski, and René Thiemann. Analyzing program termination and complexity automatically with aprove. *Journal of Automated Reasoning*, 58(1):3–31, 2017. `doi:10.1007/s10817-016-9388-y`.

24. Ashutosh Gupta, Thomas A. Henzinger, Rupak Majumdar, Andrey Rybalchenko, and Ru-Gang Xu. Proving non-termination. In *Proceedings of the 35th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '08, page 147–158, New York, NY, USA, 2008. Association for Computing Machinery. doi:10.1145/1328438.1328459.
25. Matthias Heizmann, Daniel Dietsch, Marius Greitschus, Jan Leike, Betim Musa, Claus Schätzle, and Andreas Podelski. Ultimate automizer with two-track proofs. In *Proceedings of the 22nd International Conference on Tools and Algorithms for the Construction and Analysis of Systems - Volume 9636*, page 950–953, Berlin, Heidelberg, 2016. Springer-Verlag. doi:10.1007/978-3-662-49674-9_68.
26. Matthias Heizmann, Jochen Hoenicke, Jan Leike, and Andreas Podelski. Linear ranking for linear lasso programs. *CoRR*, abs/1401.5347, 2014. URL: http://arxiv.org/abs/1401.5347, arXiv:1401.5347.
27. Jera Hensel, Constantin Mensendiek, and Jürgen Giesl. Aprove: Non-termination witnesses for c programs - (competition contribution). In *International Conference on Tools and Algorithms for Construction and Analysis of Systems*, 2022. URL: https://api.semanticscholar.org/CorpusID:247858517.
28. Daniel Larraz, Kaustubh Nimkar, Albert Oliveras, Enric Rodríguez-Carbonell, and Albert Rubio. Proving non-termination using max-smt. In Armin Biere and Roderick Bloem, editors, *Computer Aided Verification*, pages 779–796, Cham, 2014. Springer International Publishing.
29. Jan Leike. Ranking function synthesis for linear lasso programs. *ArXiv*, abs/1401.5351, 2014. URL: https://api.semanticscholar.org/CorpusID:8593497.
30. Jan Leike and Matthias Heizmann. Ranking templates for linear loops. *Log. Methods Comput. Sci.*, 11(1), 2015. doi:10.2168/LMCS-11(1:16)2015.
31. Jan Leike and Matthias Heizmann. Geometric nontermination arguments. *CoRR*, abs/1609.05207, 2016. URL: http://arxiv.org/abs/1609.05207, arXiv:1609.05207.
32. Ravindra Metta, Hrushikesh Karmarkar, Kumar Madhukar, R. Venkatesh, and Supratik Chakraborty. PROTON: PRObes for termination or not (competition contribution). In *TACAS 2024*, volume 14572 of *LNCS*, pages 393–398. Springer, 2024. doi:10.1007/978-3-031-57256-2_27.
33. Ravindra Metta, Umang Mathur, and Saurabh Joshi. PROTON 2.1: Synthesizing ranking functions via fine-tuned locally hosted LLM (competition contribution). In *TACAS 2025*, volume 14682 of *LNCS*, pages 407–411. Springer, 2025.
34. Arkadi Nemirovski and Kees Roos. *Motzkin transposition theoremMotzkin Transposition Theorem*, pages 2345–2348. Springer US, Boston, MA, 2009. doi:10.1007/978-0-387-74759-0_405.
35. Andreas Podelski and Andrey Rybalchenko. A complete method for the synthesis of linear ranking functions. In Bernhard Steffen and Giorgio Levi, editors, *Verification, Model Checking, and Abstract Interpretation*, pages 239–251, Berlin, Heidelberg, 2004. Springer Berlin Heidelberg.
36. Caterina Urban. The Abstract Domain of Segmented Ranking Functions. In Logozzo, Francesco, Fähndrich, and Manuel, editors, *Lecture Notes in Computer Science*, volume 7935 of *Lecture Notes in Computer Science*, pages 43–62, Seattle, United States, June 2013. Springer. URL: https://inria.hal.science/hal-00925670, doi:10.1007/978-3-642-38856-9_5.
37. Shaowei Zhu and Zachary Kincaid. Reflections on termination of linear loops. In Alexandra Silva and K. Rustan M. Leino, editors, *Computer Aided Verification*, pages 51–74, Cham, 2021. Springer International Publishing.

38. Shaowei Zhu and Zachary Kincaid. Termination analysis without the tears. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, PLDI 2021, page 1296–1311, New York, NY, USA, 2021. Association for Computing Machinery. doi:10.1145/3453483.3454110.